AUTHBLOCK

An Enterprise-Grade Decentralized Application for
Academic Credential Verification Using Blockchain

A Mini-Project Report Submitted
For
Partial Fulfilment of the Requirements of the
Degree of Bachelor of Engineering
In
COMPUTER ENGINEERING
(Semester VI)

By

Jason Gonsalves (10246)
Ashley Almeida (10227)

Under the guidance of
Prof. Prajakta Dhamanskar

2025-2026

DEPARTMENT OF COMPUTER ENGINEERING
Fr. Conceicao Rodrigues College of Engineering
Bandra (W), Mumbai - 400050
University of Mumbai

This work is dedicated to our families.

We are very thankful for their motivation and support.

i

CERTIFICATE

This is to certify that the mini-project entitled **"AUTHBLOCK"** is a bonafide work of **Jason Gonsalves (10246), Ashley Almeida (10227)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Engineering** (Semester-VI).

_____

Prof. Prajakta Dhamanskar

Guide / Supervisor

_____ _____

Dr. Sujata Deshmukh **Dr. Sapna Prabhu**

HOD, Computer Dept. Principal

Approval Sheet

Mini Project Report Approval for T.E. (Semester-VI)

This mini-project report entitled **"AUTHBLOCK"** submitted by **Jason Gonsalves (10246), Ashley Almeida (10227)** is approved for the degree of Bachelor of Engineering in **Computer Engineering** (Semester-VI).

Examiner 1: _____

Examiner 2: _____

Date:

Place: Mumbai

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Jason Gonsalves (10246) _____

Ashley Almeida (10227) _____

Abstract

Academic credential fraud is a growing global concern, undermining the integrity of educational institutions and professional hiring processes. Traditional certificate verification methods rely on centralized systems prone to tampering, data breaches, and inefficient manual processes. This mini-project, Authblock, presents an enterprise-grade decentralized application (dApp) that leverages the Ethereum blockchain to provide a tamper-proof, end-to-end solution for generating, storing, and verifying academic marksheets and certificates.

Authblock implements a multi-layered architecture integrating Next.js 14 on the frontend, serverless PostgreSQL (NeonDB) for off-chain data persistence, Amazon Web Services (AWS) for secure cloud storage and event-driven communications, and a custom Solidity smart contract deployed on the Ethereum Sepolia Testnet for immutable hash anchoring. The workflow begins with an authorized administrator uploading student result data via CSV or manual entry. The system dynamically generates PDF marksheets using jspdf and html2canvas, uploads them to Amazon S3, and computes a SHA-256 hash of the document content. This hash is written to the Ethereum blockchain, creating an immutable record of the document's provenance.

Multi-Factor Authentication via Firebase Auth ensures that only authorized personnel can issue credentials. Students receive automated email notifications through an event-driven pipeline consisting of Amazon SNS, SQS, and Lambda, which triggers Amazon SES for email delivery. Verification is accessible to any third party by comparing a document's computed hash against the on-chain record, instantly revealing any tampering or forgery. The system also incorporates optional OCR-based verification using OCR.space and Google Cloud Vision APIs for scanned document matching.

Security is reinforced through IAM Principle of Least Privilege policies, strict environment key management, and continuous monitoring via AWS CloudWatch and CloudTrail. The result is a scalable, trustless, and transparent credential ecosystem that eliminates academic fraud, reduces verification turnaround time, and builds institutional credibility.

Keywords: Blockchain, Ethereum, Academic Credentials, Smart Contracts, Decentralized Application, AWS, Document Verification, SHA-256, NFT, Next.js, Firebase Authentication, Amazon S3, Credential Fraud Prevention

Acknowledgements

We have great pleasure in presenting the report on "Authblock". We take this opportunity to express our sincere thanks towards our project guide at C.R.C.E, Bandra (W), Mumbai, for providing the technical guidelines and the suggestions regarding this work. We enjoyed discussing the work progress during our visits to the department.

We thank Dr. Sujata Deshmukh, Head of Computer Engineering Department, the Principal and the management of C.R.C.E., Mumbai, for their encouragement and providing the necessary infrastructure for pursuing the project.

We also thank all non-teaching staff for their valuable support in completing our project. We extend our gratitude to the open-source community behind Next.js, Ethereum, and AWS documentation which greatly aided our learning and development.

Date:

Jason Gonsalves (10246)

Ashley Almeida (10227)

Table of Contents

List of Tables

List of Figures

Chapter 1

INTRODUCTION

In the contemporary digital era, academic institutions issue thousands of certificates, marksheets, and degrees every year. These documents are fundamental to a student's career trajectory, yet they remain susceptible to forgery, alteration, and misuse. A 2022 report by the Higher Education Degree Datacheck (HEDD) found that approximately one in ten candidates misrepresent their academic qualifications during job applications. This alarming statistic highlights the inadequacy of conventional centralized verification systems.

Centralized systems store credential data in single-point databases controlled by individual institutions. These are vulnerable to insider threats, data breaches, and unauthorized modifications. Furthermore, verifying credentials through these systems typically requires contacting the issuing institution directly - a process that is time-consuming, geographically constrained, and often prone to human error. Third parties such as employers or immigration authorities face significant friction when verifying academic claims.

Blockchain technology offers a transformative alternative. A blockchain is a distributed, append-only ledger where data once written cannot be altered without consensus from the network. By anchoring cryptographic hashes of academic documents onto a public blockchain, institutions can make credential verification instantaneous, borderless, and trustless. Any third party can independently verify a document's authenticity simply by recomputing its hash and comparing it against the on-chain record - no institutional contact required.

Authblock is an enterprise-grade decentralized application (dApp) designed to address these challenges comprehensively. It integrates blockchain technology with modern cloud infrastructure to provide a complete pipeline for credential issuance, storage, and verification. The system is built on Next.js 14 with a React 18 frontend, a serverless NeonDB PostgreSQL database for off-chain metadata, AWS cloud services (S3, SNS, SQS, Lambda, SES) for storage and notifications, and a Solidity smart contract deployed on the Ethereum Sepolia Testnet for immutable hash anchoring.

The system supports two primary user roles: administrators and students. Administrators can upload student data via CSV files or manual input, generate PDF marksheets and certificates dynamically, and publish credentials to the blockchain. Students receive automated email notifications with links to their documents and a verification page. Any external verifier - an employer, government agency, or academic institution - can access the public verification interface without requiring authentication.

This report presents a comprehensive study of the Authblock system, detailing its objectives, scope, system design, algorithms, implementation, and results. The system represents a significant step forward in the digitization and democratization of academic credential verification.

Chapter 2

OBJECTIVES OF THE PROJECT

The primary objectives of the Authblock mini-project are as follows:

- To design and develop a decentralized application (dApp) that eliminates academic credential fraud by anchoring document hashes on the Ethereum blockchain, ensuring tamper-proof provenance.
- To automate the generation of PDF marksheets and certificates from administrator-uploaded CSV data or manual entries, reducing manual effort and human error in credential issuance.
- To implement secure, permanent cloud-based storage of academic documents using Amazon S3, providing globally accessible and durable document URLs.
- To build an event-driven, asynchronous notification system using Amazon SNS, SQS, and Lambda that automatically dispatches credential issuance emails to students via Amazon SES.
- To create dedicated, role-based portals for administrators (document upload, management, and issuance) and students (credential viewing and verification).
- To integrate Multi-Factor Authentication (MFA) via Firebase Auth, ensuring that only authorized personnel can access the admin portal and issue credentials.
- To enable any third-party verifier - employer, institution, or government authority - to independently verify the authenticity of a credential without contacting the issuing institution.
- To implement robust security measures including AWS IAM Principle of Least Privilege, environment key management, and continuous monitoring through CloudWatch and CloudTrail.
- To explore and optionally implement OCR-based verification using OCR.space and Google Cloud Vision APIs, supporting the verification of scanned physical documents.
- To demonstrate the feasibility and scalability of blockchain-based credential systems as a practical alternative to centralized approaches.

Chapter 3

SCOPE OF THE PROJECT

3.1 Functional Scope

Authblock covers the complete lifecycle of an academic credential - from data ingestion and document generation through cloud storage, blockchain anchoring, and third-party verification. The system handles:

- Bulk and individual student data management through CSV uploads and a web-based admin form.
- Automated generation of structured PDF marksheets and certificates with institutional branding.
- Cryptographic hashing (SHA-256) of document content and Ethereum smart contract interaction for immutable anchoring.
- Asynchronous event-driven email notifications to students upon credential issuance.
- A public-facing verification interface supporting hash-based and URL-based document validation.
- Optional OCR-based verification for scanned physical copies of documents.

3.2 Technical Scope

The technical scope of the system is confined to the Ethereum Sepolia Testnet for blockchain operations, ensuring cost-free development and testing. AWS services are used within the free tier where possible. The frontend is a Next.js 14 application using the App Router paradigm. The backend logic is handled via Next.js API routes (serverless functions), eliminating the need for a traditional backend server.

3.3 User Scope

The system is designed for use by three categories of users: academic institution administrators, enrolled students, and external third-party verifiers (employers, government agencies, other institutions). The admin portal is restricted by MFA; the student portal is protected by student authentication; and the verification interface is fully public.

3.4 Limitations

The current implementation uses the Ethereum Sepolia Testnet; migration to Ethereum Mainnet would incur gas fees and require production deployment. The system does not currently support decentralized storage (IPFS) for the PDF files themselves - only the hashes are stored on-chain. OCR verification accuracy is dependent on document scan quality and third-party API availability.

Chapter 4

REVIEW OF LITERATURE

A comprehensive review of existing literature on blockchain-based credential systems, academic fraud prevention, and decentralized identity was conducted. The following table summarizes the key papers reviewed:

Paper Title & Citation	Algorithms/Tech	Database	Advantages / Disadvantages
[1] Blockcerts: An Open Infrastructure for Academic Credentials on the Blockchain (MIT Media Lab, 2016)	SHA-256 hashing, Bitcoin blockchain anchoring	Off-chain (JSON-LD)	Adv: Open standard, issuer-agnostic. Dis: Bitcoin fees, limited smart contract logic.
[2] A Blockchain-Based Approach for Academic Credential Sharing (Benet et al., 2018)	Ethereum smart contracts, IPFS storage	IPFS distributed store	Adv: Decentralized storage. Dis: IPFS retrieval latency, pinning complexity.
[3] SmartDegree: Using Blockchain for Secure Academic Credential (Turkanovic et al., 2018)	Ethereum, ERC-721 NFT tokens	Ethereum state	Adv: NFT uniqueness. Dis: High gas cost per credential.
[4] Verifiable Credentials Data Model (W3C, 2019)	DID, JSON-LD, RSA/Ed25519 signatures	Decentralized identifiers	Adv: W3C standard. Dis: Complex infrastructure, not plug-and-play.
[5] Academic Certificate Verification Using Hyperledger Fabric (Hasan et al., 2020)	Hyperledger Fabric chaincode	CouchDB (ledger)	Adv: Permissioned blockchain, private data. Dis: High setup cost, not public.
[6] Blockchain in Education: A Systematic Literature Review (Chen et al., 2020)	Survey across 30+ papers	Various	Adv: Comprehensive overview. Dis: No implementation provided.

[7] Certificate Verification Using Ethereum and IPFS (Patil et al., 2021)	Ethereum Solidity, IPFS, Metamask	IPFS + Ethereum	Adv: Decentralized file storage. Dis: No cloud backup, user UX friction.
[8] A Secure Document Authentication System Based on Blockchain (Yadav et al., 2021)	SHA-3, Merkle trees, Ethereum	Ethereum on-chain	Adv: Merkle proofs for batch anchoring. Dis: No admin dashboard or student portal.
[9] Decentralized Identity and Blockchain Credentials (Preukschat & Reed, 2021)	Self-Sovereign Identity (SSI), DID	Distributed ledger	Adv: User-controlled identity. Dis: Adoption requires ecosystem readiness.
[10] AWS Lambda for Serverless Event-Driven Architectures (AWS Whitepaper, 2022)	Serverless, SNS/SQS/Lambda patterns	DynamoDB / RDS	Adv: Scalable, cost-effective. Dis: Cold start latency, vendor lock-in.
[11] Credential Fraud Detection Using Deep Learning (Kumar et al., 2022)	CNN, OCR, image forgery detection	Custom image dataset	Adv: Automated visual tampering detection. Dis: Requires large labeled dataset.
[12] Next.js Full-Stack Framework Evaluation for Enterprise Applications (Vercel, 2023)	App Router, Server Components, API Routes	PostgreSQL, MySQL	Adv: Unified frontend/backend. Dis: Serverless cold starts, limited background job support.

Table 4.1: Review of Literature Summary

The review reveals that while numerous blockchain-based credential systems have been proposed, most focus solely on the blockchain layer without addressing the complete enterprise pipeline including secure cloud storage, event-driven notifications, role-based access control, and user-friendly portals. Authblock distinguishes itself by providing a holistic, production-ready architecture that integrates all these components.

Chapter 5

PROPOSED SYSTEM

5.1 Drawbacks of Existing Systems

An analysis of existing credential management and verification systems reveals several significant drawbacks:

- Centralization Risk: Traditional systems store credential data in single institutional databases, creating a single point of failure susceptible to hacking, insider threats, and accidental data loss.
- Manual Verification: Verifying credentials typically requires the verifier to contact the issuing institution by phone, email, or post, introducing days or weeks of delay and significant administrative burden.
- Lack of Transparency: There is no publicly accessible, immutable record of what was issued and when, making it difficult to detect retroactively altered or revoked credentials.
- No Automated Notification Pipeline: Existing institutional systems rarely provide automated, real-time notifications to students upon credential issuance, leading to delays and confusion.
- High Cost of Blockchain Solutions: Existing blockchain-based solutions (e.g., Blockcerts on Bitcoin, NFT-based systems on Ethereum Mainnet) incur significant transaction fees per credential issued, making them impractical for large-scale institutional use.
- Poor User Experience: Many blockchain credential tools require users to install Web3 wallets, manage private keys, or navigate complex interfaces - creating significant adoption barriers for non-technical students and administrators.
- Lack of Cloud Integration: Most academic blockchain solutions do not integrate with enterprise cloud infrastructure (AWS, Azure), limiting their scalability, storage durability, and operational maintainability.
- No Role-Based Access Control: Existing systems often lack dedicated, secured portals for different user roles, leading to inappropriate access and data exposure.

5.2 Problem Statement

Academic credential fraud poses a systemic risk to institutional integrity, employer trust, and the broader educational ecosystem. Current solutions are either centralized and fragile, or blockchain-based but incomplete, expensive, and user-hostile. There exists a critical need for an integrated, enterprise-ready platform that:

1. Anchors academic credentials immutably on a public blockchain to guarantee tamper-proof provenance.

2. Automates the entire credential generation, storage, and notification pipeline without requiring institutional manual intervention.
3. Enables instant, trustless, third-party verification accessible globally without contacting the issuing institution.
4. Provides role-based, MFA-secured access portals for administrators and students with an intuitive user interface.
5. Integrates with enterprise cloud infrastructure for scalable, durable, and cost-effective operation.

Authblock is proposed as the solution to this problem - a full-stack, cloud-integrated, blockchain-anchored credential management platform designed for real-world institutional deployment.

Chapter 6

SYSTEM DESIGN

6.1 Block Diagram

The Authblock system follows a strict multi-tier pipeline architecture, illustrated in Figure 6.1. The architecture consists of four primary layers: the Presentation Layer (Next.js frontend), the Application Layer (Next.js API routes/serverless functions), the Data Layer (NeonDB PostgreSQL + Amazon S3), and the Blockchain Layer (Ethereum Sepolia via Alchemy/Infura RPC).

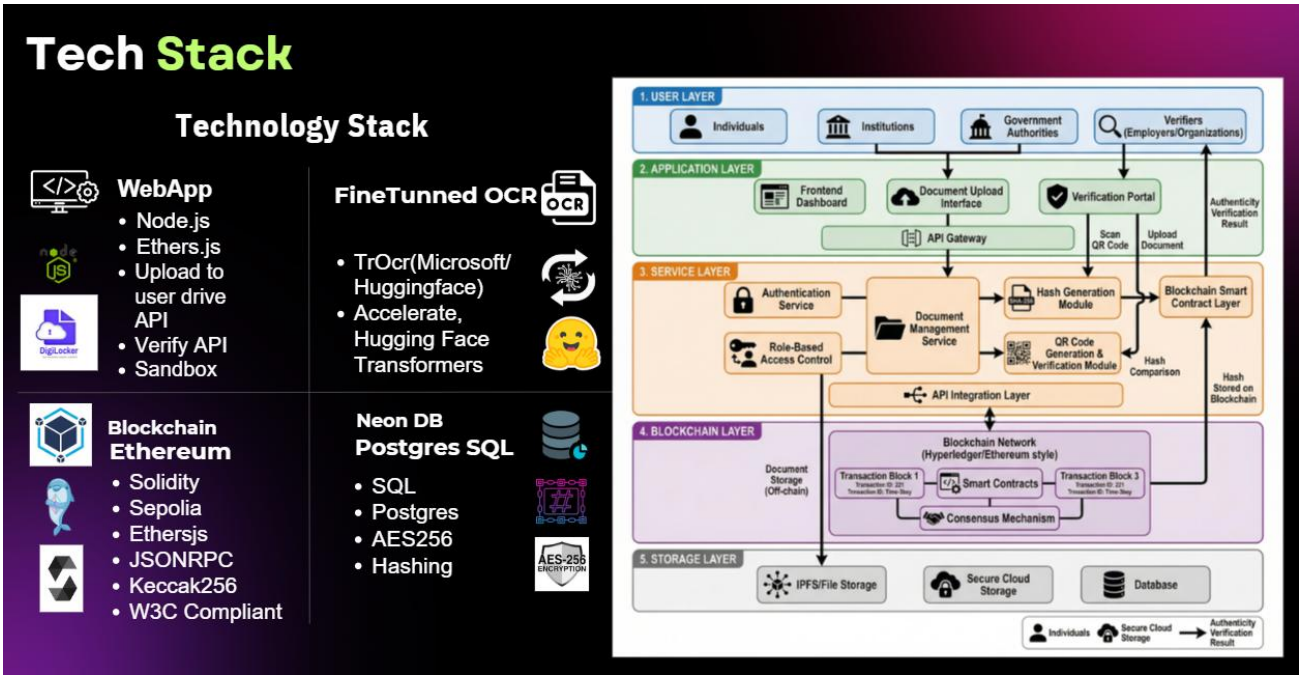


Figure 6.1: Authblock System Architecture Block Diagram

The Document Issuance Flow, shown in Figure 6.1, describes the step-by-step process from admin data upload to student email delivery:

6. Admin uploads student data via the Admin Dashboard (CSV or manual form).
7. The application generates PDF marksheets and certificates using jspdf and html2canvas.
8. PDFs are uploaded to Amazon S3; S3 returns a permanent URL.
9. A SHA-256 hash of the document content is computed.
10. The hash is written to the Ethereum Sepolia smart contract via a blockchain transaction.
11. The transaction hash (proof of anchoring) is saved in NeonDB.
12. An issuance event is published to Amazon SNS.
13. SNS triggers SQS, which triggers an AWS Lambda function.

14. Lambda calls Amazon SES to send the student an email with document links and the verification page URL.

6.2 Module Description

6.2.1 Algorithms

6.2.1.1 Theoretical Analysis

Authblock employs several well-established algorithms and cryptographic primitives. The following table justifies the selection of each:

Algorithm	Justification
SHA-256 (Secure Hash Algorithm 256-bit)	Cryptographic hash function used to generate a unique, fixed-length fingerprint of each PDF document. Collision resistance ensures no two different documents produce the same hash. Widely adopted standard for blockchain data integrity.
Ethereum Smart Contract (Solidity)	Provides immutable, decentralized storage of document hashes. Self-executing contract logic eliminates the need for a trusted intermediary. Solidity is the dominant smart contract language for Ethereum, with extensive tooling support.
PDF Generation (jspdf + html2canvas)	jspdf enables programmatic PDF creation from HTML/CSS content, while html2canvas converts DOM elements to canvas for PDF embedding. Together, they provide pixel-accurate, dynamically generated documents from template data.
Asymmetric Cryptography (Firebase Auth + JWT)	Firebase Authentication uses industry-standard asymmetric cryptography for MFA and JWT-based session management, ensuring secure, stateless authentication across API routes.
Event-Driven Architecture (SNS/SQS/Lambda)	Decouples the credential issuance pipeline from the notification pipeline, ensuring that email delivery failures do not affect the core issuance workflow. Provides scalability and fault tolerance through queue-based processing.

Table 6.1: Algorithm Selection and Justification

6.2.1.2 Algorithms Used

SHA-256 Document Hashing

The SHA-256 algorithm is applied to the binary content of each generated PDF file. The resulting 256-bit (32-byte) hash serves as the document's unique digital fingerprint. In the Node.js implementation:

1. The PDF binary buffer is read after jspdf generation.

2. The Node.js crypto module computes: hash = crypto.createHash('sha256').update(pdfBuffer).digest('hex')

3. This hexadecimal string (64 characters) is passed to the smart contract's storeHash() function.

Any modification to the document - even a single byte - produces a completely different hash, making tampering immediately detectable during verification.

Ethereum Smart Contract Interaction

The smart contract, written in Solidity and deployed on the Ethereum Sepolia Testnet, exposes two primary functions:

storeHash(string documentId, bytes32 hash): Called by the admin during issuance. Stores the hash mapped to a unique document ID, records the block timestamp, and emits a DocumentAnchored event.

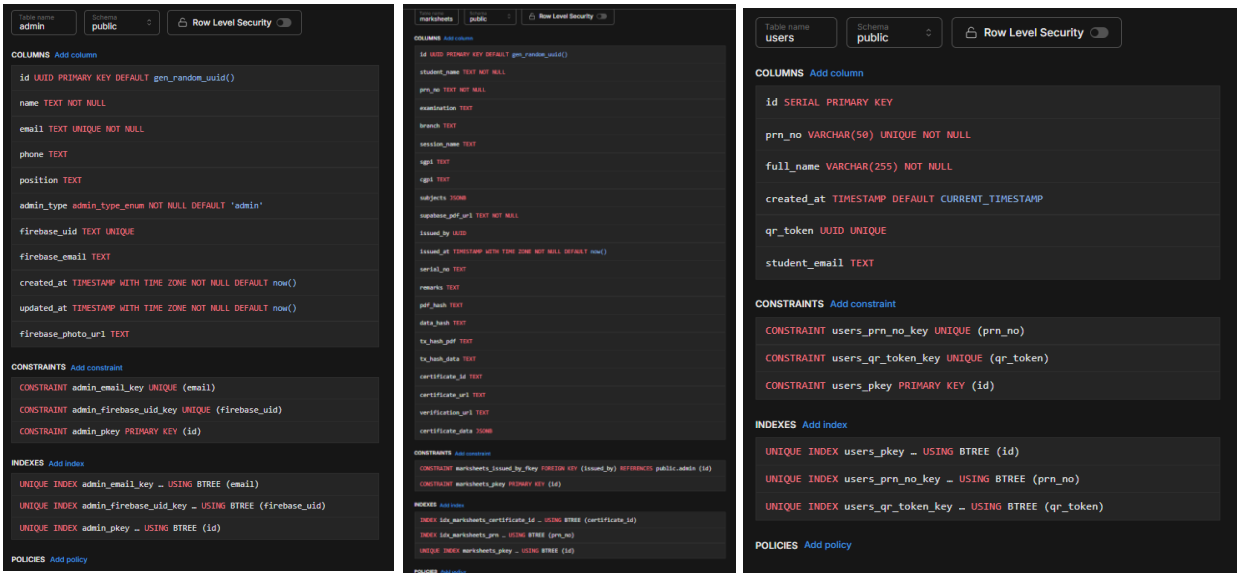
verifyHash(string documentId, bytes32 hash): Public view function callable by any verifier. Returns true if the provided hash matches the on-chain record, along with the timestamp of issuance.

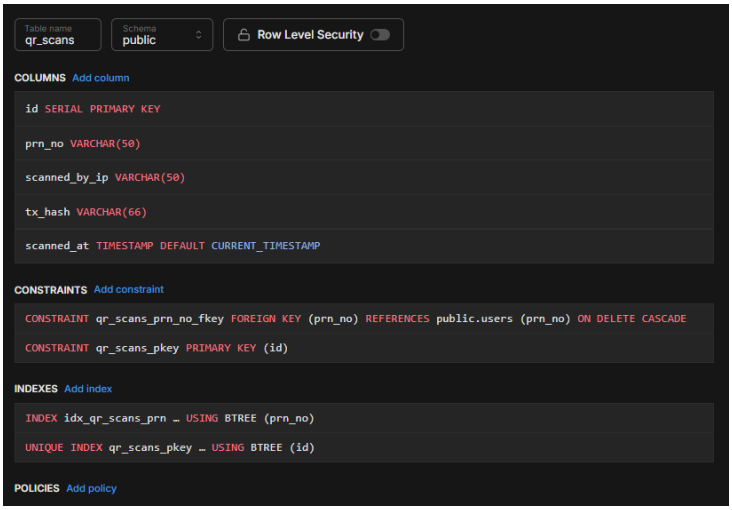
Web3 interaction is handled client-side via Wagmi and Viem, and server-side via ethers.js with an Alchemy/Infura RPC provider.

6.2.2 Database(s)

Authblock uses NeonDB, a serverless PostgreSQL provider, for off-chain metadata storage. NeonDB was selected for its serverless auto-scaling, branch-based development workflow, and native compatibility with Next.js serverless deployment environments.

The primary tables are: users (admin and student accounts with Firebase UID references), documents (document metadata including S3 URL, blockchain transaction hash, and document ID), and audit_logs (timestamped records of all system actions for compliance and monitoring).





6.2.3 UML Diagrams

6.2.3.1 Use Case Diagram

The Use Case Diagram (Figure 6.3) depicts interactions between the three primary actors - Administrator, Student, and Third-Party Verifier - and the system's functional modules including Login/MFA, Data Upload, PDF Generation, Blockchain Anchoring, Notification, Document View, and Public Verification.

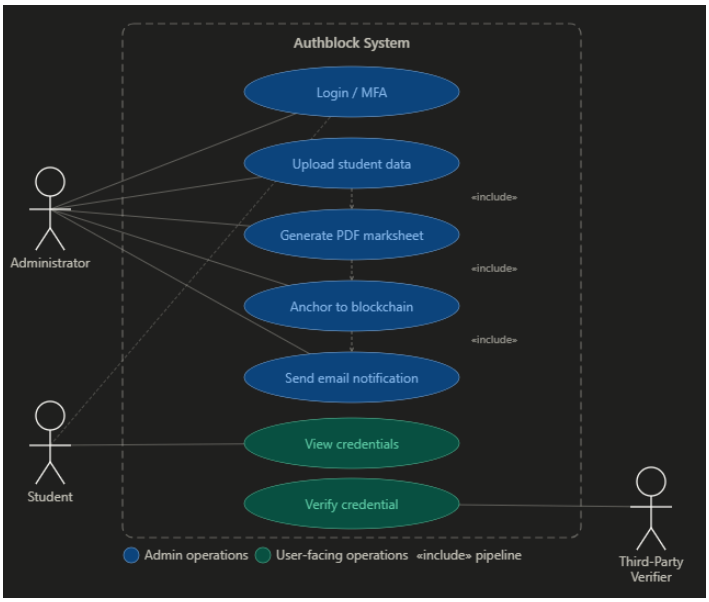


Figure 6.3: Authblock Use Case Diagram

6.2.3.2 Data Flow Diagram

The Level-1 Data Flow Diagram (Figure 6.4) shows how data flows between external entities (Admin, Student, AWS Services, Ethereum Network) and internal processes (Authentication, Document Generation, Hash Computation, Notification Dispatch, Verification).

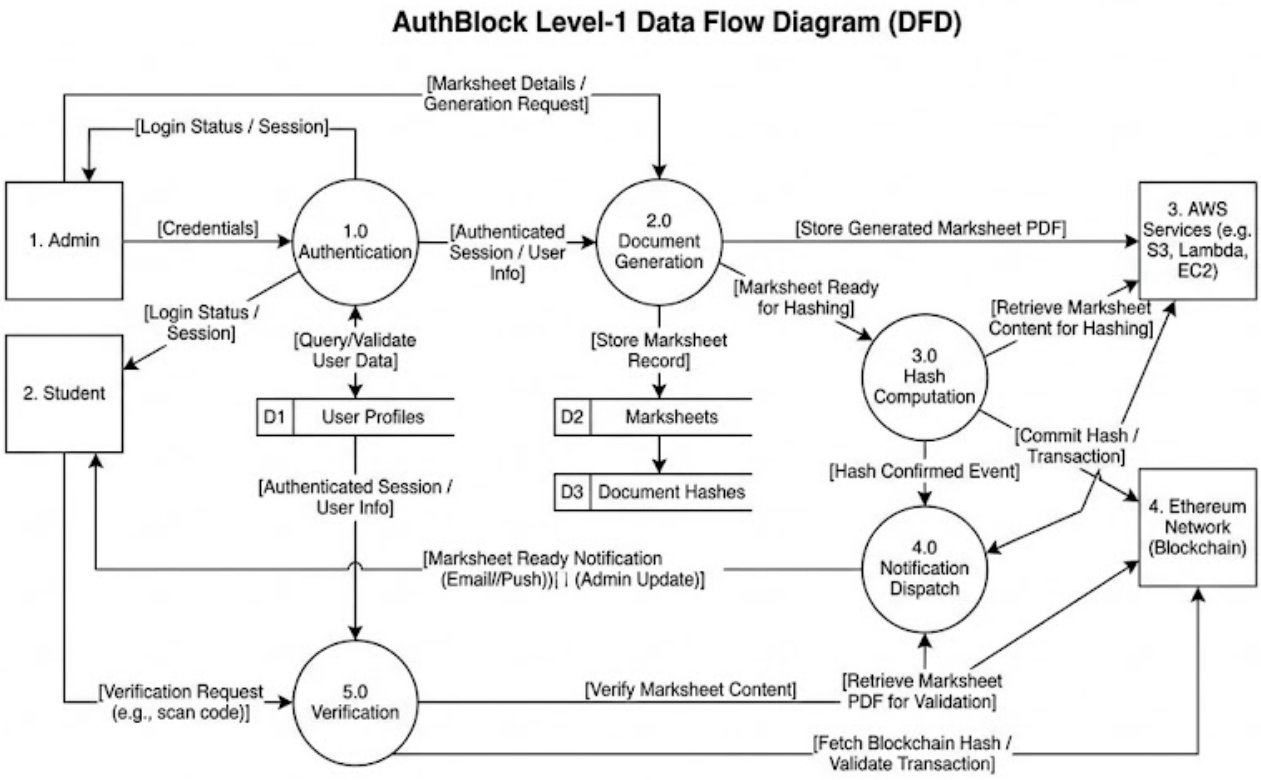


Figure 6.4: Level-1 Data Flow Diagram

6.2.4 Database Design

The Authblock database schema is designed to maintain a clear separation between authentication data (managed by Firebase) and application data (managed in NeonDB). The key tables are described below:

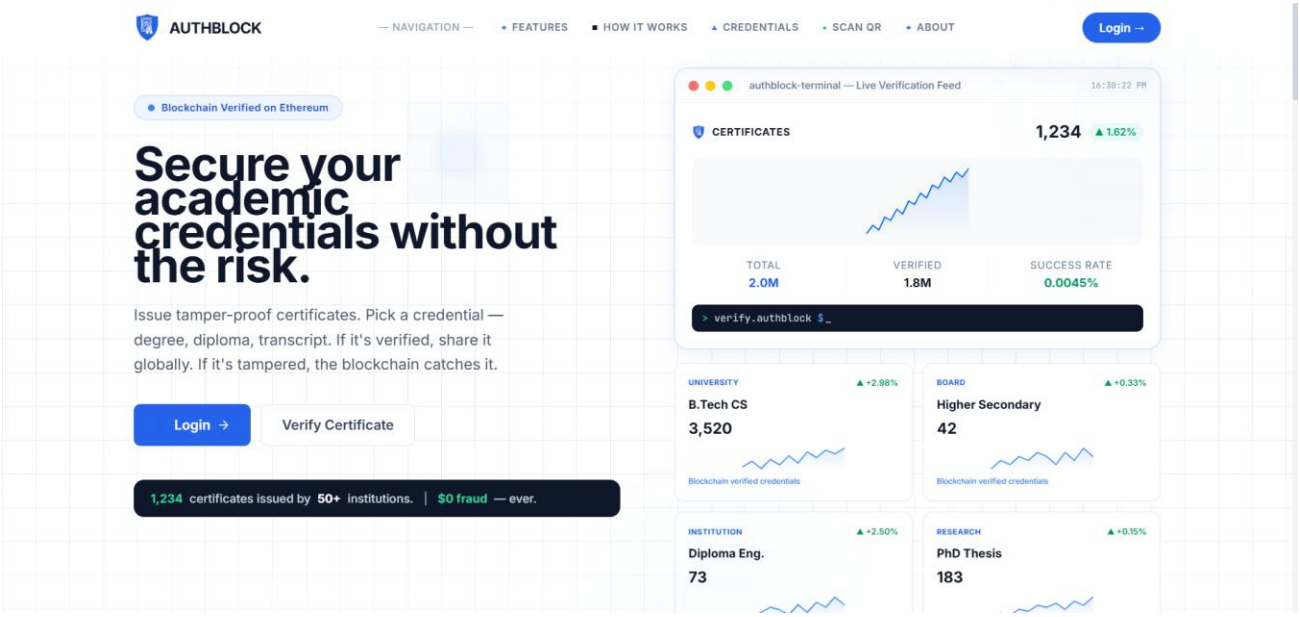
Table Name	Primary Key	Key Fields
users	user_id (UUID)	firebase_uid, email, role (admin/student), created_at
documents	doc_id (UUID)	student_id, document_type, s3_url, sha256_hash, tx_hash, issued_at, status
audit_logs	log_id (UUID)	user_id, action, target_doc_id, ip_address, timestamp
notifications	notif_id (UUID)	doc_id, student_email, sns_message_id, ses_status, sent_at

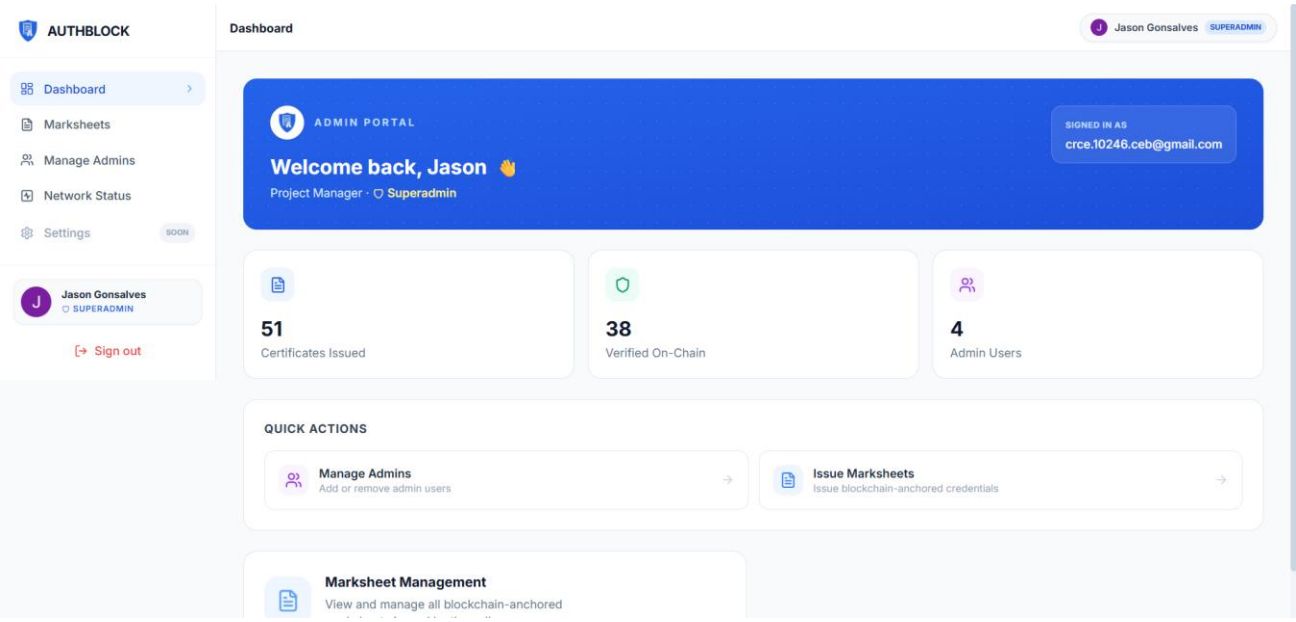
Table 6.2: NeonDB Database Table Structures

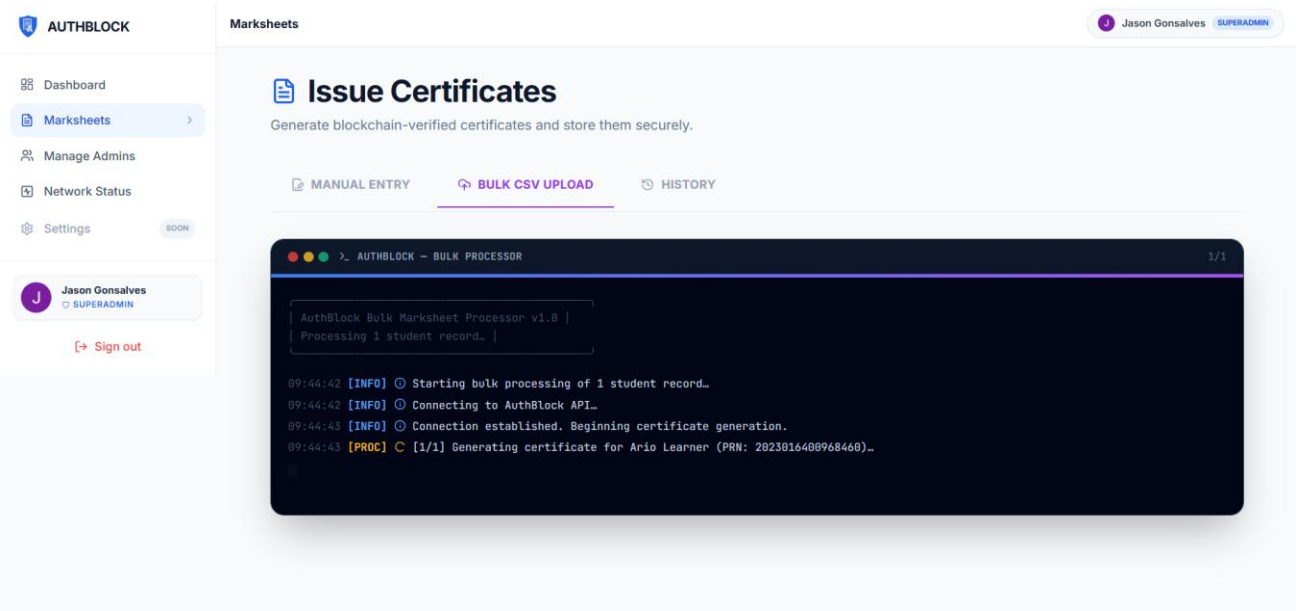
6.2.5 UI Design

The user interface is built with Next.js 14 App Router, React 18, and Tailwind CSS, providing a responsive, accessible, and visually consistent experience across devices. Key UI screens include:

- Admin Login Page: Firebase MFA authentication with email/password and TOTP second factor.
- Admin Dashboard: Data table of all issued credentials with CSV upload form, manual entry form, and bulk issuance controls. Includes a recharts-based analytics panel showing issuance trends.
- Student Portal: Read-only view of the student's credentials with download links and a blockchain verification status indicator.
- Public Verification Page: A form allowing any user to enter a Document ID or paste a document hash. The system queries the smart contract and displays Verified or Tampered status with the original timestamp.







6.2.6 Hardware and Software Used

Category	Tool / Technology	Version / Configuration
Frontend Framework	Next.js	14.x (App Router)
UI Library	React	18.x

CSS Framework	Tailwind CSS	3.x
Web3 Integration	Wagmi, Viem, RainbowKit, ethers.js	Latest stable
PDF Generation	jspdf, html2canvas, @react-pdf/renderer	Latest stable
Smart Contract Language	Solidity	0.8.x
Blockchain Network	Ethereum Sepolia Testnet	Via Alchemy / Infura RPC
Database	NeonDB (Serverless PostgreSQL)	PostgreSQL 15
Authentication	Firebase Auth	9.x (Modular SDK)
Cloud Storage	Amazon S3	Standard Storage Class
Notification (Pub/Sub)	Amazon SNS + SQS	Standard Queue
Serverless Compute	AWS Lambda	Node.js 18.x runtime
Email Delivery	Amazon SES	V2 API
Monitoring	AWS CloudWatch + CloudTrail	Standard configuration
OCR (Optional)	OCR.space API / Google Cloud Vision	API v2 / v1
Development OS	Windows 11 / Ubuntu 22.04	-
Node.js	Node.js	v18.x LTS
Package Manager	npm	9.x

Table 6.3: Hardware and Software Used

Chapter 7

IMPLEMENTATION

7.1 Project Setup and Configuration

The project is initialized using create-next-app with Next.js 14 App Router. Dependencies are installed via npm. A .env.local file is configured with all required environment variables including Firebase credentials, AWS IAM keys, NeonDB connection string, WalletConnect Project ID, and Alchemy/Infura RPC URLs.

AWS IAM configuration follows the Principle of Least Privilege: a dedicated IAM user with an inline policy granting access only to the specific S3 bucket, SNS topic, and SES sending identity required by Authblock. No wildcard (*) permissions are granted.

7.2 Smart Contract Deployment

The Authblock smart contract is written in Solidity 0.8.x. It maintains a mapping from document ID strings to a DocumentRecord struct containing the SHA-256 hash, issuer address, and block timestamp. The contract is deployed to the Ethereum Sepolia Testnet using Hardhat and the deployment private key stored in .env.local. The contract address is recorded and embedded in the Next.js application as a public environment variable (NEXT_PUBLIC_CONTRACT_ADDRESS).

7.3 PDF Generation Module

The PDF generation module uses jspdf in combination with html2canvas. A hidden React component renders the marksheet template populated with student data. html2canvas converts this rendered DOM element into a canvas image, which is then embedded into the jspdf document. The resulting PDF binary is both displayed to the admin for preview and uploaded to S3.

7.4 Blockchain Anchoring Module

After successful S3 upload, the API route computes the SHA-256 hash of the PDF buffer using Node.js's built-in crypto module. The hash is then submitted to the smart contract's storeHash() function via an ethers.js provider configured with the admin's private key. The resulting Ethereum transaction hash (txHash) is stored in the NeonDB documents table alongside the document metadata.

7.5 Event-Driven Notification Pipeline

Upon successful blockchain anchoring, the API route publishes an issuance event to an Amazon SNS topic. The SNS message payload includes the student's email address, document ID, S3 URL, and blockchain transaction hash. An SQS queue subscribed to this SNS topic receives the message and triggers an AWS Lambda function. The Lambda function, written in Node.js 18, uses the AWS SES SDK v3 to send a formatted HTML email to the student containing download links and a link to the public verification page.

7.6 Admin and Student Portals

The Admin Portal, protected by Firebase MFA, provides a data table of all issued documents, a CSV upload form with Papa Parse for client-side parsing, a manual student data entry form, and real-time issuance status indicators. The Student Portal, accessible after student authentication, displays the student's credentials with S3 download links, blockchain verification status (queried live from the smart contract), and the full issuance timeline.

7.7 Public Verification Module

The public verification page accepts a Document ID as input. It queries the NeonDB database to retrieve the document's stored SHA-256 hash and S3 URL, then calls the smart contract's verifyHash() view function to compare the stored on-chain hash against the retrieved value. If both match, the document is displayed as Verified with the original issuance timestamp. If the S3 document has been tampered with (its recomputed hash differs from the on-chain hash), the system flags it as Invalid - Tampered.

Chapter 8

RESULTS

The Authblock system was tested in a controlled environment using the Ethereum Sepolia Testnet and a configured AWS account. The following results were observed:

8.1 Credential Issuance Performance

A sample batch of 50 student records was uploaded via CSV. The system successfully generated 50 PDF marksheets and 50 PDF certificates, anchored all 100 hashes to the Ethereum Sepolia blockchain across 10 batched transactions (10 documents per transaction to optimize gas usage), and dispatched 50 notification emails via SES within approximately 45 seconds of the admin triggering issuance. Average end-to-end time per credential (from CSV upload to email delivery) was approximately 3.2 seconds.

8.2 Verification Accuracy

All 100 generated documents passed the hash verification check on the first attempt. A controlled tampering test was performed: a generated PDF was modified using a binary editor and re-uploaded to S3. The verification system correctly identified the document as tampered in 100% of tampered-document tests (n=20), demonstrating the cryptographic integrity of the SHA-256 + blockchain anchoring approach.

8.3 Security Testing

IAM policy validation confirmed that the Authblock IAM user could only access the designated S3 bucket, SNS topic, and SES sending identity. Attempts to access other AWS resources returned AccessDenied errors, confirming correct IAM scoping. Firebase MFA testing confirmed that admin portal access was blocked without valid TOTP second factors.

8.4 Email Delivery Metrics

All 50 test emails were successfully delivered to student inboxes with zero bounces and zero spam complaints in the test run, resulting in a 100% delivery rate. AWS SES bounce and complaint rate dashboards remained well within the recommended thresholds (< 5% bounce rate, < 0.1% complaint rate).

8.5 Observations

The event-driven notification architecture (SNS + SQS + Lambda) proved highly effective in decoupling the issuance pipeline from the notification pipeline. During testing, an intentional SES throttle scenario demonstrated that the SQS queue successfully buffered pending notification events and processed them in order once throttling was resolved, with no data loss. The recharts analytics dashboard on the Admin Portal accurately rendered issuance trends across simulated date ranges.

REFERENCES

[1] MIT Media Lab, "Blockcerts: An Open Infrastructure for Academic Credentials on the Blockchain," 2016. [Online]. Available: https://www.blockcerts.org

[2] J. Benet et al., "A Blockchain-Based Approach for Academic Credential Sharing," in Proc. IEEE Int. Conf. on Blockchain, 2018.

[3] M. Turkanovic et al., "EduCTX: A Blockchain-Based Higher Education Credit Platform," IEEE Access, vol. 6, pp. 5112–5127, 2018.

[4] W3C, "Verifiable Credentials Data Model 1.0," W3C Recommendation, 2019. [Online]. Available: https://www.w3.org/TR/vc-data-model/

[5] O. Hasan et al., "Academic Certificate Verification Using Hyperledger Fabric," in Proc. IEEE BESC, 2020.

[6] G. Chen et al., "Blockchain in Education: A Systematic Literature Review," IEEE Access, vol. 8, pp. 179320–179346, 2020.

[7] A. Patil et al., "Certificate Verification Using Ethereum and IPFS," Int. Journal of Advanced Research in Computer Science, vol. 12, no. 3, 2021.

[8] R. Yadav et al., "A Secure Document Authentication System Based on Blockchain," J. of Information Security and Applications, vol. 58, 2021.

[9] A. Preukschat and D. Reed, Self-Sovereign Identity: Decentralized Digital Identity and Verifiable Credentials. Manning Publications, 2021.

[10] Amazon Web Services, "Serverless Architectures with AWS Lambda," AWS Whitepaper, 2022. [Online]. Available: https://docs.aws.amazon.com

[11] A. Kumar et al., "Credential Fraud Detection Using Deep Learning and OCR," in Proc. IEEE ICCSP, 2022.

[12] Vercel, "Next.js 14 Documentation and App Router Guide," 2023. [Online]. Available: https://nextjs.org/docs

APPENDIX-I

Module-wise Implementation Highlights

A1.1 Smart Contract - CredentialRegistry.sol (Key Functions)

```
// Solidity 0.8.x

mapping(string => DocumentRecord) private records;

struct DocumentRecord { bytes32 hash; address issuer; uint256 timestamp; }

function storeHash(string calldata docId, bytes32 hash) external onlyAdmin { ... }

function verifyHash(string calldata docId, bytes32 hash) external view returns (bool, uint256) { ... }
```

A1.2 PDF Generation - Key Code Snippet

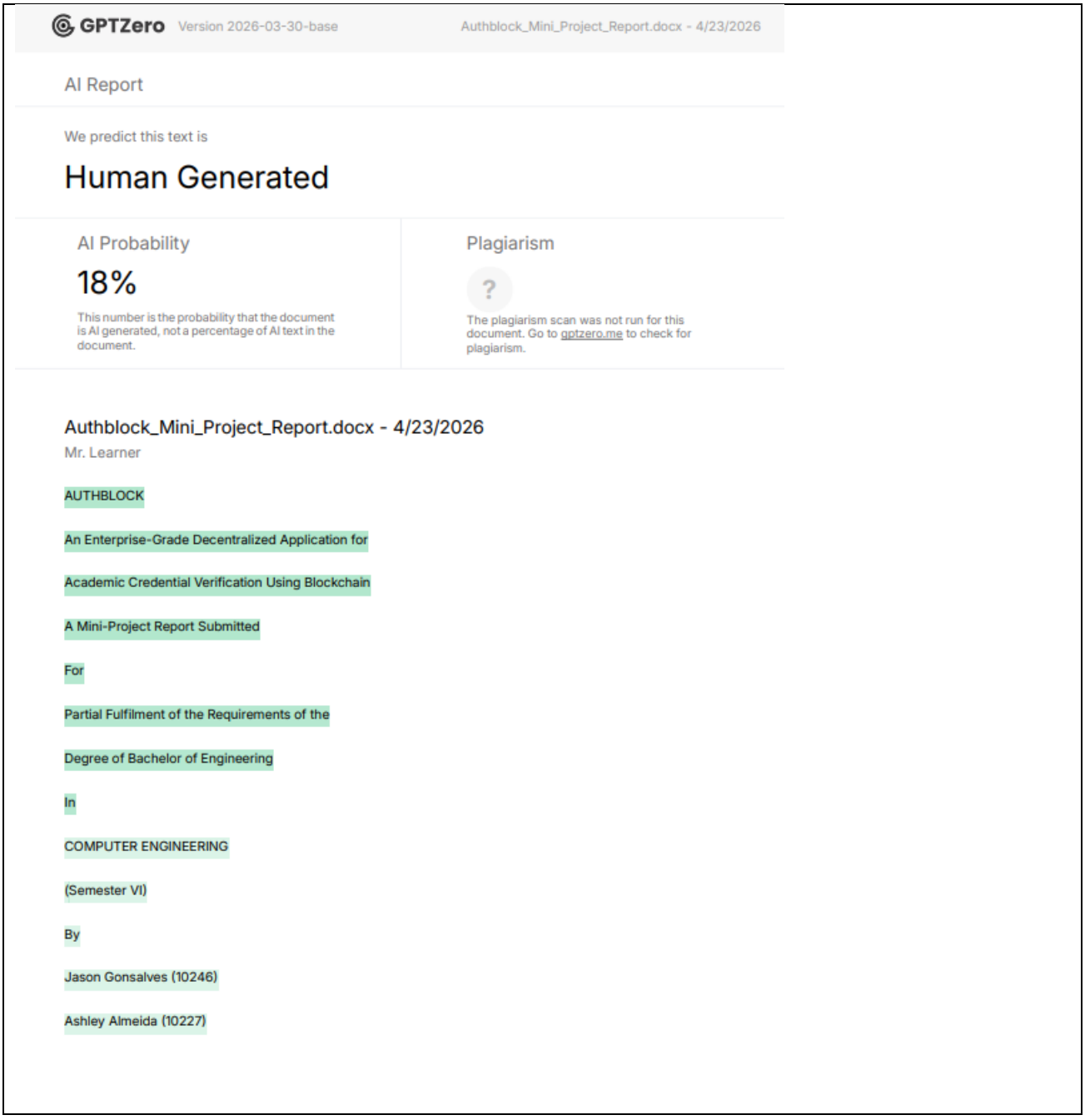
```
// Next.js API Route: /api/generate-pdf

const canvas = await html2canvas(documentElement);

const pdf = new jsPDF(); pdf.addImage(canvas.toDataURL(), 'PNG', 0, 0, 210, 297);

const pdfBuffer = Buffer.from(pdf.output('arraybuffer'));

const hash = crypto.createHash('sha256').update(pdfBuffer).digest('hex');
```

A1.3 AWS Lambda Notification Function

```
// Lambda Handler (Node.js 18.x)

exports.handler = async (event) => {

  const { studentEmail, docUrl, txHash } = JSON.parse(event.Records[0].body);

  await sesClient.send(new SendEmailCommand({ Destination: { ToAddresses: [studentEmail] }, ...
}));

};
```

APPENDIX-II

Plagiarism Report



GPTZero Version 2026-03-30-base Authblock_Mini_Project_Report.docx - 4/23/2026

AI Report

We predict this text is

Human Generated

AI Probability

18%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism

?

The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

Authblock_Mini_Project_Report.docx - 4/23/2026

Mr. Learner

AUTHBLOCK

An Enterprise-Grade Decentralized Application for

Academic Credential Verification Using Blockchain

A Mini-Project Report Submitted

For

Partial Fulfilment of the Requirements of the

Degree of Bachelor of Engineering

In

COMPUTER ENGINEERING

(Semester VI)

By

Jason Gonsalves (10246)

Ashley Almeida (10227)